

Situs staging dan enam checkpoint pengembangan

Kato Report adalah sistem pelaporan kondisi toilet kampus UPI Tasikmalaya. Mahasiswa melapor lewat QR, AI mengubah keluhan menjadi tiket kerja, dan penyelesaiannya dibuktikan dengan foto yang diperiksa AI. Dokumen ini menjelaskan bagaimana sistem itu dibangun dalam enam checkpoint, dan bagaimana setiap checkpoint bisa dicoba langsung di situs staging.

Situs: `ai-complaint-staging.yvrtz.workers.dev`

Repo: `rdnsa/AI-complaint`, branch `staging`

Daftar isi

1. Ringkasan
2. Cara memakai situs staging
3. Arsitektur staging (teknis)
4. Checkpoint 1: Pondasi
5. Checkpoint 2: Sesuai kampus
6. Checkpoint 3: Bukti, jejak, dan akun
7. Checkpoint 4: Rapi dan tajam
8. Checkpoint 5: AI kedua
9. Checkpoint 6: Siap pakai
10. Rekap perkembangan
11. Operasional staging
12. Lampiran: daftar commit

1. Ringkasan

Apa yang dibangun, dan bagaimana progresnya dilaporkan.

Masalah. Keluhan tentang toilet kampus jarang sampai ke petugas kebersihan. Mahasiswa tidak tahu harus melapor ke siapa, dan tidak ada cara untuk memastikan keluhannya ditindaklanjuti. Akibatnya masalah kecil dibiarkan sampai menjadi besar.

Solusi. Stiker QR di setiap lantai. Mahasiswa memindai QR dan menulis keluhan dengan bahasa sehari-hari. AI (DeepSeek) menggolongkan keluhan, menentukan prioritas, dan menyusun perintah kerja. Petugas menutup laporan dengan foto bukti yang diperiksa AI kedua (Google Gemini). Supervisor memantau semuanya lewat dashboard dan Chatbot AI.

Cara melaporkan progres. Sistem final ada di branch `main`. Riwayat pengembangannya dibagi menjadi enam checkpoint, masing-masing ditandai dengan tag git `checkpoint-1` sampai `checkpoint-6`. Setiap checkpoint dijalankan sebagai aplikasi tersendiri di situs staging, sehingga siapa pun bisa mencoba sistem persis seperti pada tahap tersebut.

#	CHECKPOINT	INTI PERUBAHAN	COMMIT	ENDPOINT API
1	Pondasi	Lapor via QR, klasifikasi dan prioritas dengan LLM, dashboard petugas, ringkasan harian	3	16
2	Sesuai kampus	Gedung UPI, QR per lantai, dwibahasa, pelacakan status, papan laporan publik	4	19
3	Bukti, jejak, akun	Foto wajib, bukti penyelesaian, log aktivitas, grafik, akun berperan, papan peringkat	2	26
4	Rapi dan tajam	Arsitektur berlapis, dashboard analitis, "Laporan saya" per akun, dokumentasi	4	26
5	AI kedua	Verifikasi foto dengan vision AI, Chatbot AI, mode gelap	5	28
6	Siap pakai	Tiga peran, petugas tanpa login, kamera langsung, laporan pekerjaan, rebrand Kato Report, failover API key	17	32

2. Cara memakai situs staging

Untuk siapa saja yang ingin mencoba setiap checkpoint.

Alamat

ALAMAT	ISI
<code>/checkpoint</code>	Hub: daftar checkpoint, fitur baru di masing-masing, dan tombol untuk mencobanya
<code>/checkpoint3</code>	Pindah ke checkpoint 3, lalu membuka berandanya
<code>/checkpoint3/peringkat</code>	Pindah ke checkpoint 3, lalu langsung membuka halaman <code>/peringkat</code>
<code>/dokumentasi</code>	Dokumen ini. Versi PDF ada di <code>/dokumentasi.pdf</code>
alamat lain	Aplikasi checkpoint yang sedang dipilih. Kalau belum memilih, yang dibuka checkpoint terbaru

Langkah mencoba

- 1 Buka `/checkpoint` , lalu pilih checkpoint yang ingin dicoba. Mulailah dari checkpoint 1 supaya perkembangannya terasa.
- 2 Penanda "**Checkpoint N/6 · ganti**" di pojok kiri bawah menunjukkan checkpoint yang sedang aktif. Ketuk penanda itu untuk kembali ke hub.
- 3 Ikuti skenario uji di bagian setiap checkpoint dalam dokumen ini.
- 4 Untuk membandingkan dua checkpoint berdampingan, buka checkpoint kedua di jendela penyamaran atau browser lain.

Akun untuk mencoba

CHECKPOINT	MAHASISWA / PELAPOR	PETUGAS	PENGELOLA
1-2	tanpa akun	nama bebas + sandi bersama (minta ke pengembang)	sama dengan petugas
3-5	tanpa akun, atau daftar sendiri di <code>/daftar</code>	akun petugas, dibuat oleh admin	admin: <code>admin</code> / <code>Admin123!</code>
6	tanpa akun, atau daftar sendiri di <code>/register</code>	tanpa login: pilih nama (nama ditambahkan supervisor)	supervisor: <code>admin</code> / <code>Admin123!</code>

Catatan keamanan. `Admin123!` adalah sandi bawaan dari migrasi database dan tercantum di repo publik. Sandi ini hanya boleh dipakai di staging. Di produksi (`main`) sandi itu wajib diganti.

Yang perlu diketahui penguji

- **Data setiap checkpoint terpisah.** Laporan yang dikirim di checkpoint 2 tidak muncul di checkpoint 3, karena struktur database tiap checkpoint berbeda. Silakan mengirim laporan percobaan sebanyak yang diperlukan.
- **Login juga terpisah.** Masuk di checkpoint 3 tidak membuat Anda ikut masuk di checkpoint 4.
- **Satu browser, satu checkpoint.** Berpindah checkpoint di satu tab ikut memindahkan tab lain di browser yang sama.
- **Fitur AI memakai layanan sungguhan.** Klasifikasi keluhan memakai DeepSeek, dan pemeriksaan foto (checkpoint 5-6) memakai Gemini. Kalau layanan itu sedang sibuk, laporan tetap tersimpan dan hasil AI bisa dianalisis ulang.
- **Ringkasan harian otomatis** (pukul 17.00 WIB) hanya berjalan di checkpoint terbaru. Di checkpoint lama, ringkasan tetap bisa dibuat manual dari dashboard.

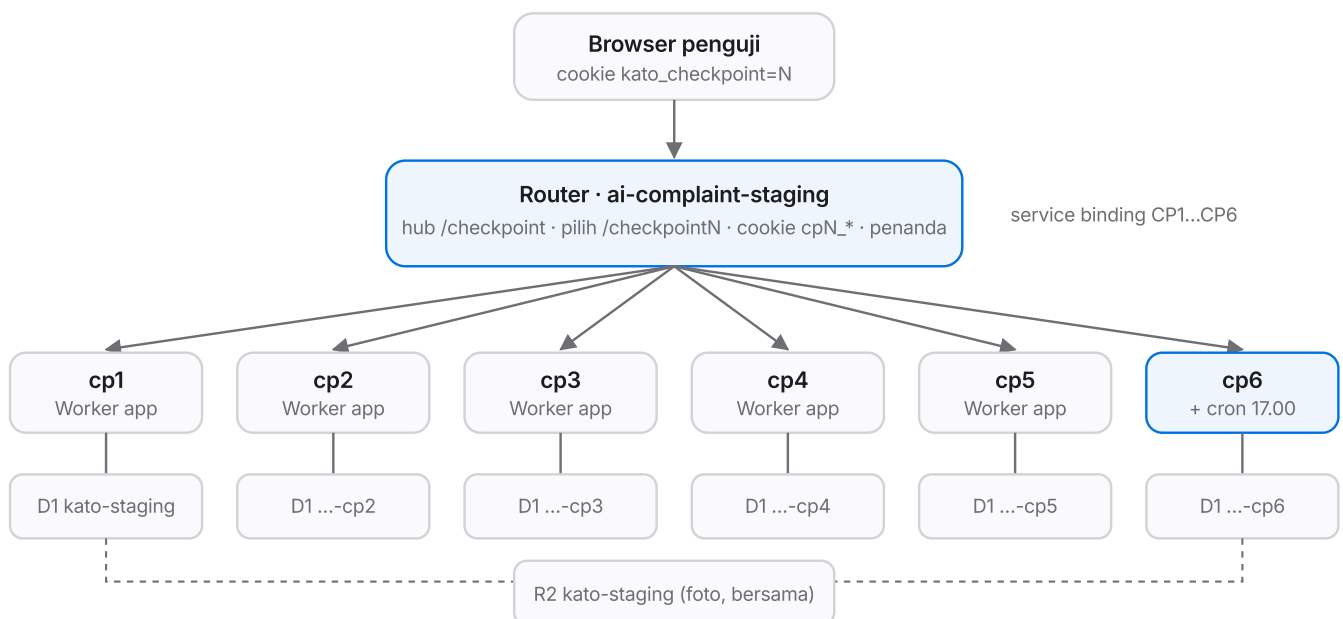
3. Arsitektur staging (teknis)

Bagaimana enam versi aplikasi berjalan bersamaan di satu domain tanpa mengubah kode aplikasinya.

Model branch dan tag

- `main` berisi sistem final, dan dari branch inilah produksi di-deploy.
- **Tag** `checkpoint-1` **sampai** `checkpoint-6` menandai commit di `main` yang menjadi batas setiap checkpoint: `dd9c91a`, `feb7e49`, `2e18c48`, `d2e7d72`, `e44958b`, dan `defb699`.
- `staging` dimulai dari `checkpoint-1`. Setiap kali melapor, tag berikutnya di-merge dengan `git merge --no-ff checkpoint-N`. Arahnya selalu satu: `main` ke `staging`, tidak pernah sebaliknya.
- File khusus staging tidak pernah ada di `main`, sehingga merge tidak pernah konflik. File-file itu adalah `STAGING.md`, `staging.config.json`, `scripts/staging.mjs`, dan folder `staging/` (`router.mjs`, `checkpoints.mjs`, `journey.mjs`, `docs/`). Setelah checkpoint 6, isi `staging` sama dengan `main` ditambah file-file ini.

Topologi



Semua permintaan masuk ke router, lalu diteruskan ke Worker checkpoint yang dipilih. Worker checkpoint tidak punya alamat publik sendiri.

Router (`staging/router.mjs`)

Aplikasi di setiap checkpoint memakai alamat absolut seperti `/api/...` dan `/lapor/...`, sehingga tidak bisa dipasang di bawah awalan seperti `/checkpoint3/` tanpa mengubah kode lamanya. Router menyelesaikan masalah itu tanpa menyentuh kode aplikasi:

1. `/checkpoint` menyajikan halaman hub statis dari binding `ASSETS`.
2. `/checkpointN[/jalur]` memasang cookie `kato_checkpoint=N` (30 hari, `SameSite=Lax`, `Secure`) lalu mengalihkan ke `/jalur`. Checkpoint yang belum dilaporkan dialihkan ke hub.

- Permintaan lain diteruskan ke Worker `ai-complaint-staging-cpN` lewat *service binding* `CPn`, sesuai cookie. Kalau cookie tidak ada, yang dipakai checkpoint terbaru. URL permintaan tidak diubah, jadi aplikasi melihat domain staging sebagai domainnya sendiri.
- Cookie aplikasi dipisah per checkpoint.** Setiap `Set-Cookie` dari aplikasi disimpan dengan awalan `cpN_`, misalnya `cp1_sesi_petugas`. Saat meneruskan permintaan, router hanya mengirim cookie berawalan checkpoint aktif dan mengembalikan namanya ke nama aslinya. Tanpa pemisahan ini, sesi dari satu checkpoint bisa diterima checkpoint lain yang memakai `AUTH_SECRET` sama, padahal id pengguna yang sama di database lain bisa berarti orang lain.
- Pada respons HTML, `HTMLRewriter` menyisipkan penanda "Checkpoint N/6 · ganti" di akhir `<body>`.

Worker, database, dan penyimpanan

SUMBER DAYA	NAMA	KETERANGAN
Worker router	<code>ai-complaint-staging</code>	Alamat publik (<code>workers.dev</code> , atau domain sendiri lewat <code>staging.config.json</code>). Tidak punya database.
Worker aplikasi	<code>ai-complaint-staging-cp1 ... -cp6</code>	Kode persis seperti di checkpoint masing-masing. <code>workers_dev</code> dan <code>preview_urls</code> dimatikan, jadi hanya bisa dicapai lewat router.
Database D1	<code>kato-staging (cp1), kato-staging-cp2 ... -cp6</code>	Satu per checkpoint, karena skemanya berubah (misalnya checkpoint 6 mengganti seluruh skema ke bahasa Inggris). Migrasi dijalankan berurutan sampai migrasi milik checkpoint itu.
Bucket R2	<code>kato-staging</code>	Dipakai bersama. Kunci objek foto berupa UUID acak, jadi tidak pernah bentrok.
Cron	hanya di checkpoint terbaru	Paket gratis Cloudflare membatasi 5 cron per akun. Cron checkpoint sebelumnya dilepas dengan <code>wrangler triggers deploy</code> .

Skrip deploy (`scripts/staging.mjs`)

Skrip ini tidak pernah mengubah `wrangler.jsonc` milik `main`. Sebagai gantinya, skrip membaca file itu dan menulis konfigurasi sementara `wrangler.staging.jsonc` yang dihapus lagi setelah selesai. Urutan kerja `node scripts/staging.mjs` :

- Menentukan checkpoint terbaru, yaitu tag terakhir yang commit-nya sudah menjadi leluhur `HEAD` (`git merge-base --is-ancestor`).

- 2 Mencari database D1 checkpoint itu, atau membuatnya (`wrangler d1 create --update-config=false`) kalau belum ada.
- 3 Menjalankan migrasi. Untuk database baru, atau dengan argumen `seed` , skrip juga mengisi data toilet dari `seed/toilets.sql` .
- 4 Men-deploy Worker checkpoint dengan nama, database, dan bucket staging, lalu mengirim secret dari `.dev.vars` .
- 5 Melepas cron dari Worker checkpoint sebelumnya.
- 6 Membuat halaman hub dan dokumentasi (`staging/journey.mjs`), lalu men-deploy router dengan service binding ke semua checkpoint yang sudah dilaporkan.

Secret

Secret Cloudflare tidak bisa dibaca kembali, jadi nilainya diambil dari `.dev.vars` lokal, yang tidak ikut di-commit. Secret yang dipakai adalah `LLM_API_KEY` , `VISION_API_KEY` , `AUTH_SECRET` , dan `PETUGAS_PASSWORD` (khusus checkpoint 1–2). Sejak checkpoint 6, `LLM_API_KEY` dan `VISION_API_KEY` boleh berisi beberapa key yang dipisah koma. Kode checkpoint yang lebih lama akan membaca daftar itu sebagai satu key yang tidak valid, jadi skrip memeriksa kodenya dulu: kalau fungsi `splitKeys` belum ada, hanya key pertama yang dikirim.



Pondasi: keluhan menjadi tiket kerja

3

commit

35

file berubah

+7.148

baris

16

endpoint API

PENJELASAN UMUM

Masalah yang dijawab. Keluhan toilet tidak punya saluran. Checkpoint ini membangun alur paling dasar: mahasiswa bisa melapor dalam hitungan detik, dan petugas menerima laporan dalam bentuk yang langsung bisa dikerjakan.

Yang bisa dilakukan

- **Mahasiswa** memindai QR di pintu WC, menulis keluhan dengan bahasa sehari-hari (bahasa gaul juga dipahami), melampirkan foto kalau mau, lalu mengirim. Konfirmasi langsung muncul.
- **AI** menentukan jenis masalah, prioritas, ringkasan yang sopan, dan perintah kerja.
- **Petugas** masuk dengan satu sandi bersama, melihat laporan dengan yang paling mendesak di atas, lalu mengubah statusnya dari *baru* ke *diproses* ke *selesai*.
- **Pimpinan** menerima ringkasan harian otomatis setiap pukul 17.00 WIB.

Contoh hasil AI

"WC lantai 2 bau banget, lantainya becek, sama sabunnya habis."

- Prioritas: **tinggi**
- Masalah: bau, genangan air, perlengkapan habis
- Perintah: keringkan lantai lebih dulu karena berisiko membuat orang terpeleset, lalu bersihkan sumber bau dan isi ulang sabun.

Skenario uji

- 1 Buka `/checkpoint1/lapor/A-1-PRIA`. Formulir sudah mengetahui WC mana yang dimaksud.
- 2 Tulis *"airnya nggenang parah, kloset yg pojok mampet"*, lalu kirim. Di halaman konfirmasi, tunggu sampai kategori dan prioritas muncul. Hasil yang diharapkan: kerusakan dan genangan air, prioritas tinggi.
- 3 Buka `/petugas`, masuk dengan nama bebas dan sandi bersama, lalu ubah status laporan tadi. Coba juga tombol analisis ulang.
- 4 Di dashboard, buat ringkasan harian secara manual.

Yang belum ada: daftar lokasi masih data contoh, pelapor tidak bisa melacak laporannya setelah menutup halaman, dan semua petugas memakai satu sandi.

Fondasi

- **Stack:** TypeScript di Cloudflare Workers dengan Hono. Database D1 (SQLite), penyimpanan foto R2, dan frontend React + Vite + Tailwind. Frontend dan API ada di **satu Worker**, sehingga tidak perlu CORS dan cukup sekali deploy. Aset statis disajikan lewat binding `ASSETS` dengan `run_worker_first: ["/api/**"]` dan fallback SPA.
- **Analisis asinkron:** `POST /api/reports` langsung menyimpan laporan dan menjawab. Analisis LLM dijalankan lewat `ctx.waitUntil()` setelah respons terkirim, sementara halaman konfirmasi melakukan polling.
- **Ketahanan:** kalau LLM gagal, laporan ditandai `ai_status = 'gagal'` beserta pesan errornya. Laporan itu tetap tampil dan bisa dianalisis ulang lewat `POST /api/reports/:id/analisa-ulang`. Latensi setiap panggilan dicatat di `ai_ms`.
- **Validasi keluaran LLM:** JSON dari DeepSeek (`response_format: json_object` , temperatur rendah, contoh *few-shot*) divalidasi dengan Zod, lalu dipaksa masuk ke enum yang dikenal database. Kategori asing dibuang, dan prioritas yang tidak dikenal menjadi *sedang*.
- **Foto lewat Worker:** `GET /api/uploads/:key` membaca objek langsung dari R2, dibatasi hanya untuk prefiks `laporan/` , dan meneruskan ETag. Alasannya, sebagian ISP di Indonesia membajak DNS untuk `*.r2.dev` , sehingga foto gagal dimuat di jaringan kampus.
- **Build otomatis:** `build.command` di `wrangler.jsonc` membangun frontend sebelum deploy, supaya `wrangler deploy` berjalan sendiri, termasuk di Workers Builds.
- **Cron** `0 10 * * *` (17.00 WIB) menyusun `daily_summaries` . Ringkasan juga bisa dibuat manual lewat `POST /api/summary/generate` .

Database: migrasi `0001_init.sql`

Tabel `toilets` (satu baris per WC fisik, `id` seperti `A-2-PRIA` di-encode ke QR), `reports` (keluhan, status, hasil AI: kategori, prioritas, ringkasan, rekomendasi, `ai_status`), dan `daily_summaries` , ditambah empat indeks untuk urutan dashboard.

Endpoint API (16)

<code>GET /api/health</code>	<code>GET /api/reports/:id</code>
<code>POST /api/auth/login</code>	<code>PATCH /api/reports/:id</code>
<code>POST /api/auth/logout</code>	<code>POST /api/reports/:id/analisa-ulang</code>
<code>GET /api/auth/me</code>	<code>POST /api/uploads</code>
<code>GET /api/toilets</code>	<code>GET /api/uploads/:key</code>
<code>GET /api/toilets/:id</code>	<code>GET /api/summary</code>
<code>POST /api/reports</code>	<code>GET /api/summary/stats</code>
<code>GET /api/reports</code>	<code>POST /api/summary/generate</code>

Halaman: `/` , `/lapor/:toiletId` , `/laporan/:id` , `/petugas` . **Konfigurasi:** vars `LLM_BASE_URL` dan `LLM_MODEL` (`deepseek-chat`); secret `LLM_API_KEY` , `AUTH_SECRET` (tanda tangan cookie sesi), dan `PETUGAS_PASSWORD` .

Sesuai kampus, dan bisa dilacak

4

commit

27

file berubah

+1.914 /

-514

baris

19

endpoint API

PENJELASAN UMUM

Masalah yang dijawab. Satu QR per WC berarti terlalu banyak stiker yang harus dicetak dan dirawat, dan daftar lokasinya belum mencerminkan kampus. Selain itu, laporan yang sudah dikirim seolah menghilang: pelapor maupun orang lain tidak bisa melihat tindak lanjutnya.

Yang berubah

- **Satu QR per lantai.** Pelapor memilih sendiri WC pria, wanita, atau disabilitas. Totalnya 13 stiker untuk 6 gedung yang punya toilet.
- **Gedung UPI Kampus Tasikmalaya yang asli**, dengan peta kampus di beranda sebagai cadangan kalau QR rusak.
- **Identitas visual kampus** (maroon, oranye bata, krem) dan pilihan bahasa Indonesia/Inggris.

Transparansi

- **Lacak laporan:** garis waktu *diterima*, *dikerjakan*, *selesai*, dan daftar "Laporan saya" di beranda.
- **Papan laporan publik** `/laporan` : semua laporan dan statusnya, tanpa login.
- **Hapus spam** oleh petugas, karena papannya kini terbuka untuk umum.

Skenario uji

- 1 Buka `/checkpoint2/lapor/A-1` , pilih jenis WC, lalu kirim keluhan.
- 2 Kembali ke beranda. Laporan tadi muncul di "Laporan saya" beserta statusnya.
- 3 Buka `/laporan` . Yang tampil hanya ringkasan AI, tanpa teks asli maupun foto pelapor.
- 4 Masuk sebagai petugas, ubah status, lalu lihat garis waktu di halaman konfirmasi ikut berubah.
- 5 Ganti bahasa ke EN dari bagian atas halaman.

PENJELASAN TEKNIS

- **Migrasi** `0002_gedung_upi.sql` : tabel `gedung` berisi sepuluh gedung, dan `toilets` mendapat `gedung_kode` . Kolom `gedung` dan `nama` dihapus, dan ditambah indeks lokasi. Id lokasi QR berformat `<kode gedung>-<lantai>` , misalnya `A-1` .

- **Seed yang aman dijalankan ulang:** semua WC dinonaktifkan dulu, lalu yang terdaftar diaktifkan kembali. Versi sebelumnya menghapus isi tabel dan gagal karena *foreign key* begitu ada laporan. Lantai yang terdaftar: A 1–3, B 1, C 1–5, G 1, H 1, dan I 1–2. Gedung D, E, F, dan J disiapkan sebagai komentar.
- **Pelacakan tetap anonim:** id laporan (UUID acak) disimpan di `localStorage`. `GET /api/reports/ringkas` mengambil status banyak laporan dalam satu permintaan. Halaman konfirmasi menyegarkan diri dengan cepat selama analisis berjalan, melambat setelahnya, dan berhenti saat tab tidak terlihat.
- **Papan publik** `GET /api/reports/publik` sengaja tidak menyertakan teks asli, foto pelapor, maupun nama petugas. Mengubah, menganalisis ulang, dan menghapus tetap membutuhkan sesi petugas.
- `DELETE /api/reports/:id` juga menghapus foto di R2 agar tidak meninggalkan berkas yatim.
- **i18n** memakai kamus sendiri tanpa pustaka. Hasil LLM tetap berbahasa Indonesia karena pembacanya petugas.

+ **endpoint:** `GET /api/lokasi`, `GET /api/lokasi/:id`, `GET /api/reports/publik`, `GET /api/reports/ringkas`, `DELETE /api/reports/:id`. – **endpoint:** `GET /api/toilets`, `GET /api/toilets/:id`.

Halaman baru: `/laporan`, dan `/lapor/:lokasiId` menggantikan `/lapor/:toiletId`.

Bukti, jejak, dan akun

2

commit

35

file berubah

+2.128 /

-178

baris

26

endpoint API

PENJELASAN UMUM

Masalah yang dijawab. Laporan bisa ditutup dengan satu klik tanpa bukti. Karena semua petugas memakai satu sandi, tidak bisa diketahui siapa melakukan apa, dan mencabut akses satu orang berarti mengganti sandi semua orang.

Akuntabilitas

- **Foto keadaan wajib** dilampirkan pelapor.
- **Selesai berarti ada foto bukti.** Tanpa foto bukti, laporan tidak bisa ditutup. Foto bukti tampil di papan publik.
- **Catatan aktivitas** merekam semua kejadian, termasuk laporan yang dihapus.
- **Grafik:** laporan masuk dan selesai per hari, prioritas, kategori, per gedung, dan rata-rata waktu penyelesaian.

Akun

- Akun pribadi dengan peran **admin**, **petugas**, atau **pelapor**.
- Admin mengelola akun petugas dan menonaktifkannya kalau perlu.
- Mahasiswa boleh mendaftar untuk masuk **papan peringatan**. Melapor tanpa akun tetap bisa.

Skenario uji

- 1 Buka `/checkpoint3/daftar`, buat akun pelapor, lalu kirim laporan dengan foto. Cek `/peringkat`.
- 2 Masuk di `/masuk` sebagai `admin` / `Admin123!`, lalu buat satu akun petugas di tab Pengguna.
- 3 Masuk sebagai petugas itu dan coba menandai laporan *selesai* tanpa foto. Hasil yang diharapkan: ditolak. Unggah foto bukti, dan laporan pun selesai.
- 4 Kembali sebagai admin, lalu buka tab Grafik dan Aktivitas.

PENJELASAN TEKNIS

- **Migrasi** `0003_bukti_dan_aktivitas.sql` : `reports.foto_selesai_key` dan tabel `aktivitas` dengan tiga indeks. Tabel `aktivitas` sengaja **tanpa foreign key** ke `reports`, karena justru laporan yang dihapus yang paling perlu terlacak. Isinya disalin ke catatan sebelum baris aslinya dihapus.

- **Migrasi** 0004_pengguna.sql : tabel pengguna (peran admin / petugas / pelapor), reports.pelapor_id , dan akun admin bawaan.
- **Aturan wajib foto dijaga di server**, bukan hanya disembunyikan di antarmuka, sehingga tidak bisa diakali lewat klien lain.
- **Kata sandi** di-hash dengan PBKDF2-SHA256, 100.000 iterasi, salt per pengguna, lewat WebCrypto. Bcrypt dan argon2 tidak tersedia di runtime Workers tanpa WebAssembly tambahan. Pesan galat untuk "akun tidak ada" dan "sandi salah" dibuat identik untuk mencegah enumerasi username. Admin tidak bisa menonaktifkan akunnya sendiri. Secret PETUGAS_PASSWORD tidak dipakai lagi.
- **Palet grafik** dipilih lewat validator: rentang terang, ambang chroma, keterpisahan untuk buta warna, dan kontras terhadap latar.

+ endpoint: POST /api/auth/masuk , /daftar , /keluar , GET /api/auth/saya , GET /api/summary/grafik , GET /api/aktivitas , GET/POST /api/pengguna , PATCH /api/pengguna/:id , GET /api/peringkat . **- endpoint:** /api/auth/login , /logout , /me .

Halaman baru: /masuk , /daftar , /peringkat .

Rapi di dalam, tajam di dashboard

4

commit

72

file berubah

+8.574 /

-1.468

baris (termasuk
dokumen)

26

endpoint API

PENJELASAN UMUM

Masalah yang dijawab. Fitur makin banyak, tetapi kode server bercampur jadi satu, sehingga setiap perubahan berisiko merusak bagian lain. Grafik juga baru menjawab "berapa banyak", padahal pimpinan butuh jawaban "kapan", "di mana", dan "apakah prioritas AI berguna".

Yang terasa oleh pengguna

- **"Laporan saya" ikut akun**, jadi bisa dilihat dari perangkat mana pun.
- **Angka utama dengan tren** dan perbandingan dengan periode sebelumnya.
- **Peta panas hari × jam** sebagai dasar jadwal ronda.
- **Matriks gedung × jenis masalah**.
- **Waktu penyelesaian per prioritas**, untuk menguji apakah prioritas AI memang bermanfaat.

Dokumen

- **Panduan umum** tanpa istilah teknis, untuk staf dan pimpinan kampus.
- **Dokumentasi teknis** untuk penilai.
- Keduanya ditulis dalam HTML dengan *print stylesheet*, sehingga satu sumber menghasilkan halaman web dan PDF.

Skenario uji

- 1 Buka `/checkpoint4/masuk` dan masuk sebagai admin. Kirim beberapa laporan dengan prioritas berbeda, lalu selesaikan sebagian.
- 2 Buka tab Grafik dan amati deretan angka utama, peta panas, matriks, dan waktu penyelesaian per prioritas.
- 3 Daftar sebagai pelapor, kirim laporan, lalu masuk dengan akun yang sama dari browser lain. "Laporan saya" tetap muncul.

PENJELASAN TEKNIS

- **Arsitektur berlapis.** Sebelumnya 37 dari 43 pernyataan SQL berada di dalam *route handler*. Sekarang ada lima lapisan dengan ketergantungan yang hanya mengarah ke dalam:
 - `domain/` : kosakata dan aturan bisnis, tanpa import Hono, D1, maupun R2
 - `repositories/` : semua SQL, satu file per agregat
 - `services/` : *use case*, tanpa Hono
 - `adapters/` : LLM, R2, hashing, sesi, dan jam
 - `routes/` : parsing, validasi, delegasi, dan format`index.ts` menjadi satu-satunya *composition root*.
- **Aturan "tutup hanya dengan bukti"** dipindah ke lapisan domain, sehingga semua pemanggil memutuskannya dengan cara yang sama.
- **Tidak ada migrasi.** Tidak ada jalur API, kolom database, atau perilaku yang berubah, kecuali "Laporan saya": `GET /api/reports/saya` menggantikan `GET /api/reports/ringkas` . Pelapor anonim tetap bisa melacak lewat tautan konfirmasi.
- **Grafik:** skala sekuensial peta panas diperiksa agar luminansnya menurun monoton (0,933 ke 0,164). Nilai nol memakai warna permukaan, supaya "tidak ada" tidak tertukar dengan "sedikit". Setiap segmen bertumpuk diberi celah 2 px.
- **Bahasa:** README ditulis ulang, dan semua komentar kode diterjemahkan ke bahasa Inggris tanpa mengubah logika.
- **Pengujian ulang setelah restrukturisasi:** analisis LLM, wajib bukti, papan publik, papan peringkat, grafik, statistik, jejak audit, ringkasan harian, dan batas wewenang ketiga peran.

AI kedua: memeriksa foto dan menjawab pertanyaan

5

commit

43

file berubah

+3.128 /

-330

baris

28

endpoint API

PENJELASAN UMUM

Masalah yang dijawab. Foto bukti wajib diunggah, tetapi foto apa pun diterima, termasuk foto lama, foto buram, atau foto yang bukan toilet. Pimpinan yang ingin tahu "gedung mana yang paling bermasalah?" juga masih harus membaca grafik sendiri.

AI pemeriksa foto

Setiap foto bukti dinilai: benar toilet? sudah bersih? Kalau belum, foto **ditolak beserta alasannya**, laporan tetap terbuka, dan petugas diminta membersihkan lagi.

- **Ditolak:** "Lantai masih basah dan ada tisu bekas."
- **Diterima:** "Kloset dan lantai terlihat bersih tanpa sampah."

Chatbot AI dan lainnya

- Pertanyaan biasa seperti "*Gedung mana yang paling banyak laporan 30 hari terakhir?*" dijawab dengan angka dari data sungguhan.
- Foto yang lolos pemeriksaan diberi tanda **diverifikasi AI** di papan publik.
- Mode gelap, ikon mata di kolom sandi, dan logo baru.

Skenario uji

- 1 Di `/checkpoint5`, kirim laporan, lalu masuk sebagai admin (atau petugas yang dibuat admin).
- 2 Selesaikan laporan dengan foto yang *bukan* toilet. Hasil yang diharapkan: ditolak dengan alasan. Ulangi dengan foto toilet yang bersih, dan laporan pun selesai.
- 3 Ajukan pertanyaan ke Chatbot AI di beranda atau di dashboard.
- 4 Coba sakelar terang/gelap.

PENJELASAN TEKNIS

- **Dua model AI.** DeepSeek tidak bisa melihat gambar, jadi foto dinilai oleh model vision terpisah lewat endpoint yang kompatibel dengan OpenAI: `VISION_BASE_URL` Gemini dengan `VISION_MODEL = gemini-3.6-flash`. `VISION_API_KEY` jatuh ke `LLM_API_KEY` kalau tidak diisi.

- **Migrasi** `0005_verifikasi_bukti.sql` : kolom `bukti_ai_hasil` (`bersih` | `kotor` | `bukan_toilet`), `bukti_ai_alasan` , `bukti_ai_model` , dan `bukti_ai_ms` . Status *selesai* hanya diberikan kalau putusannya `bersih` .
- **Chatbot** (`GET/POST /api/tanya`) memakai *tool calling*. Model tidak pernah melihat database, hanya definisi alat: hitung laporan, waktu penyelesaian, daftar laporan, dan ringkasan harian. Hasil setiap alat berupa agregat kecil, sehingga biaya per pertanyaan tetap kecil berapa pun jumlah laporannya. Loop dibatasi beberapa putaran, lalu model dipaksa menjawab. Kuotanya 50 pertanyaan per hari untuk seluruh kampus dan 20 per IP tanpa akun. Nama petugas hanya diberikan kepada pengguna yang login.
- **Anggaran token LLM** dinaikkan ke 2.000 agar model *reasoning* tidak terpotong.
- **Tema** memakai konteks React dengan kelas `.dark` dan pilihan yang diingat di browser.

+ **endpoint:** `GET /api/tanya` , `POST /api/tanya` . **Konfigurasi baru:** vars `VISION_BASE_URL` dan `VISION_MODEL` , secret `VISION_API_KEY` .

Dirancang ulang untuk pemakaian sehari-hari

17

commit

171

file berubah

+15.332 /

-6.211

baris

32

endpoint API

PENJELASAN UMUM

Masalah yang dijawab. Untuk pemakaian sehari-hari masih ada hambatan. Petugas enggan login sambil membawa alat pel. Foto dari galeri membuka celah memakai foto lama. Pekerjaan rutin petugas di luar keluhan tidak terlihat. Dan satu beranda untuk semua orang terasa membingungkan.

Alur baru

- **Tiga pintu di beranda:** Saya mahasiswa, Saya petugas, dan Saya supervisor.
- **Petugas tanpa login:** cukup memilih nama dari daftar "Siapa kamu?". Hanya supervisor yang login.
- **Foto langsung dari kamera**, bukan dari galeri.
- **Laporan pekerjaan** untuk ronda rutin, juga dengan foto yang diperiksa AI.
- **Satu QR, dua peran**, dengan sakelar mahasiswa/petugas di halaman lantai.

Penyempurnaan

- Filter waktu pada laporan dan laporan pekerjaan.
- Peta kampus yang bisa diperbesar dan digeser, dan foto yang bisa diperbesar.
- Chatbot AI dan cetak stiker QR pindah ke beranda.
- **Kato Report:** nama dan tampilan baru. Alamat halaman kini berbahasa Inggris, tetapi alamat lama di stiker tetap berfungsi.
- Cadangan API key otomatis kalau satu key kena limit.

Skenario uji

- 1 Buka `/checkpoint6/supervisor`, masuk sebagai `admin` / `Admin123!`, lalu tambahkan satu nama petugas.
- 2 Di beranda pilih **Saya mahasiswa**, tentukan gedung dan lantai lewat peta, lalu kirim keluhan dengan foto dari kamera.
- 3 Pilih **Saya petugas**, pilih nama tadi, buka lantainya, lalu tekan *Saya kerjakan sekarang*. Setelah itu tekan *Sudah beres*, *foto buktinya*, dan lihat putusan AI.
- 4 Kirim satu laporan pekerjaan ronda rutin.
- 5 Kembali ke dashboard supervisor. Coba filter waktu, lihat laporan pekerjaan per petugas, dan ajukan pertanyaan ke Chatbot AI.

6 Buka alamat lama `/lapor/A-1` . Halaman itu dialihkan ke `/report/A-1` .

PENJELASAN TEKNIS

- **Migrasi** `0006_laporan_pekerjaan.sql` : tabel `pekerjaan` untuk laporan pekerjaan petugas, diindeks per petugas, toilet, dan waktu.
- **Migrasi** `0007_peran_spv.sql` : peran menjadi `spv` / `petugas` / `pelapor` , dan `admin` diubah menjadi `spv` . Baris petugas kini cukup berisi nama: `username` dan sandi menjadi opsional (`UNIQUE` di SQLite mengizinkan banyak `NULL`). Tabel dibangun ulang lewat salinan karena SQLite tidak bisa mengubah `CHECK` .
- **Migrasi** `0008_english_schema.sql` : seluruh skema diganti ke bahasa Inggris dengan pola *create new, copy, rename*, tanpa kehilangan data. Peran menjadi `supervisor` / `staff` / `reporter` , dan tabel aktivitas menjadi `activity_log` .
- **Frontend baru:** halaman `StudentHome` , `StaffHome` , `StaffFloor` , dan `SupervisorDashboard` . Komponen baru antara lain `Camera` (kamera langsung), `StaffPicker` (pilihan nama disimpan di `localStorage`), `RoleSwitch` , `WorkLogPanel` , `TimeFilter` , `ZoomableImage` , `ImageViewer` , `QrCodesPanel` , dan peta interaktif. Alamat lama (`/lapor` , `/petugas` , `/laporan` , `/masuk` , `/spv` , dan lainnya) dialihkan lewat `LegacyRedirect` .
- **API diganti ke bahasa Inggris** dan diperluas dengan `/api/work-logs` , `/api/staff` , `/api/reports/open` , dan `/api/reports/mine` .
- **Desain:** token warna dibaca dari variabel CSS (terang di `:root` , gelap di `.dark`), dengan gaya ala halaman produk Apple dan mode terang sebagai bawaan.
- **Failover API key:** `LLM_API_KEY` dan `VISION_API_KEY` boleh berisi beberapa key yang dipisah koma. Respons 429 (limit) atau 401/403 (key ditolak) dicoba ulang dengan key berikutnya, dan error terakhir dilaporkan kalau semua key gagal.
- **Dokumen:** BPMN *to-be*, diagram arsitektur, penjelasan diagram, dan PRD, dengan skrip `npm run docs` untuk membuat PDF-nya.

+ **endpoint (nama baru/baru):** `/api/auth/login|register|logout|me` , `/api/locations[:id]` , `/api/reports/public|open|mine` , `/api/reports/:id/reanalyze` , `GET/POST /api/work-logs` , `GET /api/staff` , `/api/summary/charts` , `/api/activity` , `/api/users[:id]` , `/api/leaderboard` , `GET/POST /api/ask` . Padanan berbahasa Indonesianya dari checkpoint 3–5 dihapus.

4. Rekap perkembangan

Perkembangan sistem dari checkpoint ke checkpoint.

Statistik per checkpoint

CHECKPOINT	COMMIT	FILE BERUBAH	BARIS +	BARIS -	ENDPOINT	MIGRASI
1 Pondasi	3	35	7.148	1	16	0001
2 Sesuai kampus	4	27	1.914	514	19	0002
3 Bukti, jejak, akun	2	35	2.128	178	26	0003, 0004
4 Rapi dan tajam	4	72	8.574	1.468	26	-
5 AI kedua	5	43	3.128	330	28	0005
6 Siap pakai	17	171	15.332	6.211	32	0006, 0007, 0008

Angka diambil dari `git diff --shortstat` antar tag. Angka checkpoint 4 dan 6 termasuk dokumen HTML/PDF yang ikut di-commit.

Evolusi kemampuan

KEMAMPUAN	1	2	3	4	5	6
Lapor via QR	per WC	per lantai	per lantai	per lantai	per lantai	per lantai
Klasifikasi dan prioritas AI	✓	✓	✓	✓	✓	✓
Foto keadaan	opsional	opsional	wajib	wajib	wajib	kamera langsung
Bukti penyelesaian	–	–	foto wajib	foto wajib	diperiksa AI	diperiksa AI
Pelacakan pelapor	–	perangkat	perangkat	akun	akun	akun
Papan laporan publik	–	✓	✓ + foto bukti	✓	✓ + verifikasi AI	✓
Masuk petugas	sandi bersama	sandi bersama	akun	akun	akun	tanpa login (pilih nama)
Pengelola	–	–	admin	admin	admin	supervisor
Dashboard	daftar	daftar	+ grafik, aktivitas	+ analitis	+ Chatbot AI	+ laporan pekerjaan, filter waktu
Bahasa antarmuka	ID	ID/EN	ID/EN	ID/EN	ID/EN	ID/EN

5. Operasional staging

Untuk pengembang yang merawat situs staging.

Persiapan sekali saja

```
npx wrangler r2 bucket create kato-staging
npx wrangler d1 create kato-staging --update-config=false # isi database_id di staging.config.json
cp .dev.vars.example .dev.vars # isi LLM_API_KEY, VISION_API_KEY, AUTH_SECRET, P
```

staging.config.json berisi nama Worker router, nama dan id database checkpoint 1, nama bucket, dan domain (opsional). Kalau domain diisi, domainnya harus berupa zone di akun Cloudflare yang sama.

Naik ke checkpoint berikutnya

```
git switch staging
git merge --no-ff checkpoint-N -m "Checkpoint N"
node scripts/staging.mjs # database dibuat dan diisi otomatis bila baru
git push origin staging
```

Perintah lain

PERINTAH	FUNGSI
node scripts/staging.mjs seed	Deploy, sekaligus mengisi ulang data toilet di database checkpoint terbaru (aman dijalankan berulang)
node scripts/staging.mjs secrets	Mengirim ulang secret dari .dev.vars ke Worker checkpoint terbaru
node scripts/staging.mjs config	Hanya menulis wrangler.staging.jsonc untuk diperiksa

Masalah yang pernah terjadi

GEJALA	PENYEBAB	PENANGANAN
Sesaat setelah deploy, checkpoint baru menjawab "Endpoint tidak ditemukan" atau penanda menunjukkan nomor lama	Router versi baru belum menyebar ke semua lokasi Cloudflare, jadi sebagian permintaan masih memakai daftar checkpoint lama	Tunggu sekitar satu menit
<code>d1 execute</code> gagal dengan <code>InternalError ... Please try again</code>	Gangguan sementara di Cloudflare saat mengimpor file SQL	Jalankan <code>node scripts/staging.mjs seed</code> lagi
<code>wrangler d1 create</code> menambahkan binding ke <code>wrangler.jsonc</code>	Wrangler menawarkan mengubah konfigurasi	Selalu pakai <code>--update-config=false</code> , dan jangan commit perubahan itu ke <code>main</code>

Batas paket gratis Cloudflare

- **D1:** 10 database per akun. Staging memakai 6, dan akun ini sudah terisi 10.
- **Cron:** 5 per akun. Staging hanya memakai 1, di checkpoint terbaru.

Membongkar staging

```
npx wrangler delete --name ai-complaint-staging
for n in 1 2 3 4 5 6; do npx wrangler delete --name ai-complaint-staging-cp$n; done
npx wrangler d1 delete kato-staging
for n in 2 3 4 5 6; do npx wrangler d1 delete kato-staging-cp$n; done
npx wrangler r2 bucket delete kato-staging    # kosongkan isinya dulu
```

Hati-hati: jangan pernah menjalankan perintah di atas dengan nama `ai-complaint` atau `kato`. Keduanya milik produksi.

6. Lampiran: daftar commit per checkpoint

Dari `git log --reverse checkpoint-(N-1)..checkpoint-N`. Commit awal `949c5f7` ("first commit") tidak dihitung.

Checkpoint 1

227dcbc Bangun sistem klasifikasi dan prioritas keluhan toilet berbasis LLM
28be726 Bangun frontend otomatis sebelum deploy
dd9c91a Sajikan foto lampiran lewat Worker, bukan domain r2.dev

Checkpoint 2

e1b3743 Terapkan identitas UPI Tasikmalaya, QR per lantai, dan dwibahasa
68a1e28 Sesuaikan daftar lokasi dengan kondisi gedung sebenarnya
37a15cc Beri pelapor cara melacak status laporannya
feb7e49 Buka daftar laporan untuk umum, tanpa membuka wewenang mengubahnya

Checkpoint 3

88ffa4a Wajibkan bukti foto, catat aktivitas, dan tampilkan grafik
2e18c48 Ganti sandi bersama dengan akun berperan, dan tambah papan peringkat

Checkpoint 4

f7d5727 Rewrite the README in English and translate all code comments
603e56f Restructure the backend into layers, and move "My reports" onto the account
c1f5bc4 Deepen the dashboard charts from counts into questions
d2e7d72 Add two documents describing the system, for two audiences

Checkpoint 5

9b2594d feat: integrate vision AI for automated verification of proof-of-completion photos
c21d18b chore: upgrade default vision model to Gemini 3.6-flash
76e303b chore: increase LLM response budget to 2000 tokens to accommodate reasoning models
7673129 docs: update documentation and generated PDFs with expanded guides and AI photo validation details
e44958b feat: add AI-powered Q&A feature for querying report analytics

Checkpoint 6

777b25c feat: add camera component for photo capture and implement staff work panel
5abc63b feat: implement time filter component and related functionality for reports and work logs
298fa53 feat: enhance header with dynamic supervisor link and accessibility features
7341294 feat: add staff and student home pages, supervisor dashboard, and staff floor functionality
39f573e refactor: update staff, student, and supervisor dashboard pages for improved readability and functionality
365a461 Refactor documentation and UI components for QR code and Chatbot AI features
ff63471 feat: add MapViewer component for interactive campus map with zoom and pan functionality

f9e4aeb feat: add MapViewer component for interactive campus map with zoom and drag functionality
89c4c71 feat: implement ZoomableImage component for enhanced image viewing experience
8aa3c08 refactor: replace 'SPV' with 'supervisor' for consistency across the application
981ea49 docs: add To-Be BPMN diagram in SVG and PNG formats
31485ee docs: add system architecture diagrams and documentation build script
b7baee9 feat: add product requirements document and update build script
1c81700 refactor: rebrand application to Kato Report and update theme color tokens
1302bf5 style: change hero links layout to vertical column in Home page
5192350 chore: update Open Graph image version and alt text metadata
defb699 feat: fail over to the next API key when one is rate-limited

Laporan progress Kato Report · branch `staging` · sumber: `staging/docs/laporan-progress.html`